

## Q. 1 — Understanding a Data Pipeline

Source: The way the data is created

Ingestion: Bring the data to the pipeline

Raw: Store the data without changes

Staging: Cleaning the data (normalize fields, correct data types, dealing with missing values, deduplicate values)

Transformation: Apply business rules

Analytics: Provide trusted, easy-to-use outputs

## Q. 2 — Compare ETL and ELT

| Area                              | ETL                                                                                                      | ELT                                                                                           |
|-----------------------------------|----------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| When transformation happens       | After the extraction and before loading the data                                                         | After extracting and loading the data                                                         |
| Where transformation usually runs | In a separate processing area                                                                            | Where the data is loaded, at the datalake/warehouse                                           |
| Suitable example                  | Data that can't be stored, for example personal clients data. First remove/encrypt this data before load | Data that you know that need keep in raw, files provided by thirds that may change the schema |

## Q. 3 — Practical: Rewrite an ETL Flow as ELT

**Extract JSON files**



**Clean and standardise with Python**



**Load cleaned rows into the warehouse**

- This is a ETL flow, before loading the data it has a cleaning stage.
- ELT flow:
  - Extract JSON files
  - ↓
  - Load row JSON files in data lake
  - ↓
  - Clean and standardise with Python
- In the data lake after loading

## Q. 4 — Scenario: Choose ETL or ELT

- ETL flow
- The data needs to be stored just with valid records, so first it is necessary to transform the date types and prices
- If there are another invalid records it can be removed before the load, for example duplicates
- Trade-off: there is no raw data if the transformations go wrong or if the original data is needed.

#### Q. 5 — MCQ: Pipeline Purpose

B. Move and process data through defined stages

#### Q. 6 — MCQ: Raw Layer

B. It stores data close to its original form

#### Q. 7 — MCQ: ETL Order

A. Extract → Transform → Load

#### Q. 8 — MCQ: ELT Transformation

B. After loading into the target platform

#### Q. 9 — MCQ: Best Fit

A. A modern warehouse can store raw data and run large SQL transformations

#### Q. 10 — Batch and Streaming Processing

| Use case                        | Choice    |
|---------------------------------|-----------|
| Daily sales report              | Batch     |
| Payment-fraud alert             | Streaming |
| Hourly stock snapshot           | Batch     |
| Immediate delivery notification | Streaming |

#### Q. 11 — Practical: Simulate Batch and Streaming Events

```
import time
order_events = [
    {"order_id":101,"amount":55.5},
    {"order_id":102,"amount":25},
    {"order_id":103,"amount":199.99},
    {"order_id":104,"amount":89.90},
```

```

        {"order_id":105,"amount":45.5}
    ]
# Batch
for order in order_events:
    print(order)
# Streaming
for order in order_events:
    time.sleep(1)
    print(order)

```

#### Q. 12 — Practical: Ingest a Customer File into Raw Storage

```

from pathlib import Path
import csv
# Folders creation
source_path = Path("D:/Nextcloud/vcortes/Vicky/Cursos/Ironhack DE
Bootcamp/Activities/Week3/Session1/data/source")
source_path.mkdir(exist_ok=True, parents=True)
raw_path = Path("D:/Nextcloud/vcortes/Vicky/Cursos/Ironhack DE
Bootcamp/Activities/Week3/Session1/data/raw")
raw_path.mkdir(exist_ok=True, parents=True)
# Create customer CSV in source
data= [
    ["customer_id", "name", "email", "city"],
    [1,"Amelia clarke", "AMELIA@EXAMPLE.COM", "Manchester"],
    [2, "Oliver Bennett", "oliver@example.com", "Bristol"]
]
customer_source_path = source_path / "customers.csv"
with open(customer_source_path, "w" , encoding="utf-8") as file:
    writer = csv.writer(file, lineterminator='\n')
    writer.writerows(data)
# Copy customer.csv from source to raw
customer_raw_path = raw_path / "customers.csv"
with open(customer_source_path, "r" ) as source_file:
    reader = csv.reader(source_file, delimiter=",")
    with open(customer_raw_path, "w" , newline='') as raw_file:
        writer = csv.writer(raw_file)
        for row in reader:
            writer.writerow(row)

```

```
with open(customer_raw_path, "r" , encoding="utf-8") as raw_file:
    reader = csv.reader(raw_file)
    print(f"Raw ingestion complete: {sum(1 for row in reader)} rows")
```

```
Raw ingestion complete: 3 rows
```

### Q. 13 — Practical: Clean Raw Customers into Staging

```
import pandas as pd
# Cleaning customer raw data
raw_customer = pd.read_csv(customer_raw_path)
raw_customer_df = raw_customer.copy()

raw_customer_df["name"] = raw_customer_df["name"].str.strip()
raw_customer_df["email"] = raw_customer_df["email"].str.lower()

raw_customer_df
# Creating staging folder
staging_path = Path("D:/Nextcloud/vcortes/Vicky/Cursos/Ironhack DE
Bootcamp/Activities/Week3/Session1/data/staging")
staging_path.mkdir(exist_ok=True, parents=True)
# Saving customer.csv cleaned file in staging
raw_customer_df.to_csv(staging_path / "customers.csv", index=False)
```

### Q. 14 — MCQ: Daily Report

A. Batch

### Q. 15 — MCQ: Immediate Alert

B. Streaming

### Q. 16 — MCQ: Ingestion

A. Moves data into a pipeline

### Q. 17 — MCQ: Staging

A. Trim names and validate required fields

### Q. 18 — MCQ: Layer Benefit

A. To preserve the original input while creating a cleaned version

### Q. 19 — Pipeline Dependencies

- If the ingest is correct then start the cleaning. Once the cleaning is done then it is transformed into the customer summary. And finally publish the report
- If clean\_customer fails the pipeline would break, and it should be an alert in order to fix the process. The transformation for the summary and the report wouldn't be processed.
- If the jobs run separately the summary would be with uncleaned data, having unstandardised data and possible invalid values.

#### Q. 20 — Practical: Stop a Pipeline after Failure

```
def ingest():
    print("Ingesting was successful")

def clean():
    if customer_raw_path.exists():
        print("Clean was successful")
    else:
        raise FileNotFoundError(f"Clean failed. {customer_raw_path} is missing")

def publish():
    print("Publishing was successful")

steps = [ingest, clean, publish]
for step in steps:
    try:
        step()
    except Exception as error:
        print(f"Pipeline stopped at {step}: {error}")
        break
else:
    print("Pipeline finished successfully")
```

#### Q. 21 — Practical: Make Ingestion Safe to Re-run